

The design of parallel programs

Moreno Marzolla
Dept. of Computer Science and Engineering (DISI)
University of Bologna

Copyright © 2013—2026
Moreno Marzolla, University of Bologna, Italy
<https://www.moreno.marzolla.name/teaching/CP/>



Except where stated otherwise, this work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). To view a copy of this license, visit <https://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

Credits

- prof. Salvatore Orlando, Univ. Ca' Foscari di Venezia
<http://www.dsi.unive.it/~orlando/>
- prof. Mary Hall, University of Utah
<https://www.cs.utah.edu/~mhall/>
- Tim Mattson, Intel

Sum-Reduction

- We start assuming a **shared-memory architecture**
 - All execution units share a common memory space
- We begin with a sequential solution and parallelize it
 - This is not always a good idea; some parallel algorithms have nothing in common with their sequential counterparts!
 - However, it is sometimes a reasonable starting point

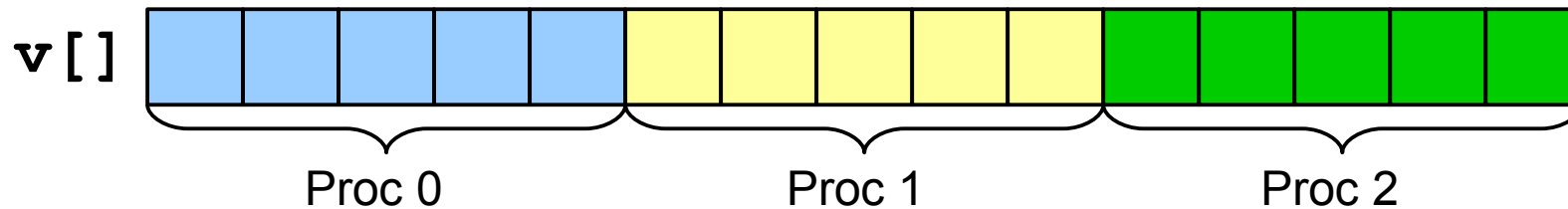
Sequential algorithm

- Compute the sum of the content of an array A of length n

```
float seq_sum(const float* v, int n)
{
    int i;
    float sum = 0.0;
    for (i=0; i<n; i++) {
        sum += v[i];
    }
    return sum;
}
```

Version 1 Shared memory (Wrong!)

- Assuming P execution units (e.g., processors), each one computes a partial sum of n / P adjacent elements
- Example: $n = 15$, $P = 3$



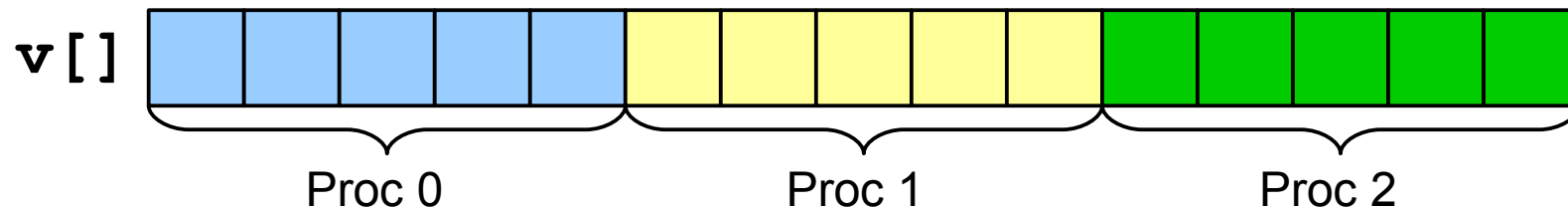
```
sum = 0.0;
do in parallel {
  my_block_len = n/P;
  my_start = my_id * my_block_len;
  my_end = my_start + my_block_len;
  for (my_i=my_start; my_i<my_end; my_i++) {
    sum += v[my_i];
  }
}
```

WRONG

Variables whose names start with *my_* are assumed to be **local** (private) to each processor; all other variables are assumed to be **global** (shared)

Version 2 (better, but still wrong)

- Assuming P processors, each one computes a partial sum of n/P adjacent elements
- Example: $n = 15$, $P = 3$

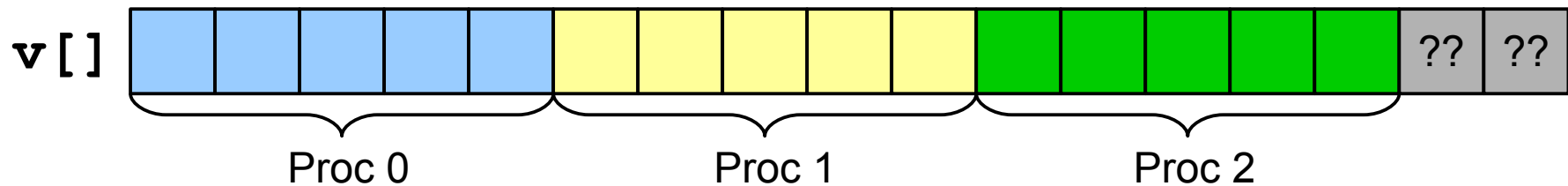


```
mutex m;  
sum = 0.0;  
do in parallel {  
    my_block_len = n/P;  
    my_start = my_id * my_block_len;  
    my_end = my_start + my_block_len;  
    for (my_i=my_start; my_i<my_end; my_i++) {  
        mutex_lock(&m);  
        sum += v[my_i];  
        mutex_unlock(&m);  
    }  
}
```

WRONG

Version 2 (better, but still wrong)

- Assuming P processors, each one computes a partial sum of n/P adjacent elements
- Example: $n = 17$, $P = 3$

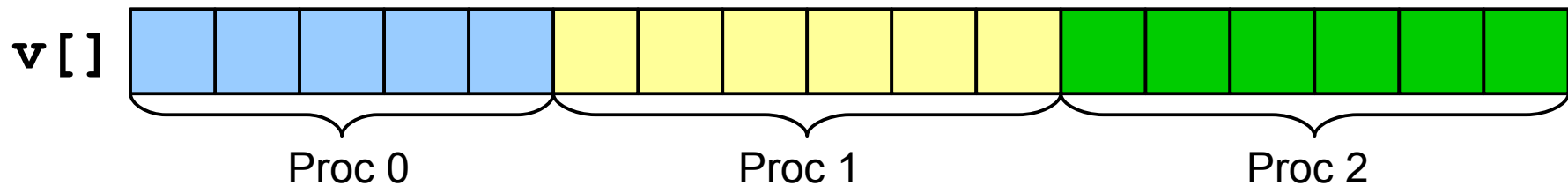


```
mutex m;  
sum = 0.0;  
do in parallel {  
    my_block_len = n/P;  
    my_start = my_id * my_block_len;  
    my_end = my_start + my_block_len;  
    for (my_i=my_start; my_i<my_end; my_i++) {  
        mutex_lock(&m);  
        sum += v[my_i];  
        mutex_unlock(&m);  
    }  
}
```

WRONG

Version 3 (correct, not efficient)

- Assuming P processors, each one computes a partial sum of n / P adjacent elements
- Example: $n = 17$, $P = 3$



```
mutex m;
sum = 0.0;
do in parallel {
    my_start = (n * my_id) / P;
    my_end = (n * (my_id + 1)) / P;
    for (my_i=my_start; my_i<my_end; my_i++) {
        mutex_lock(&m);
        sum += v[my_i];
        mutex_unlock(&m);
    }
}
```

Version 4

- Too much contention on mutex *m*
- Solution: increase the mutex *granularity*
 - Each processor accumulates the partial sum on a local (*private*) variable
 - The mutex is used at the end to update the *global* sum

```
mutex m; sum = 0.0;
do in parallel {
    my_start = (n * my_id) / P;
    my_end = (n * (my_id + 1)) / P;
    my_sum = 0.0;
    for (my_i=my_start; my_i<my_end; my_i++) {
        my_sum += v[my_i];
    }
    mutex_lock(&m);
    sum += my_sum;
    mutex_unlock(&m);
}
```

Version 5: Remove the mutex

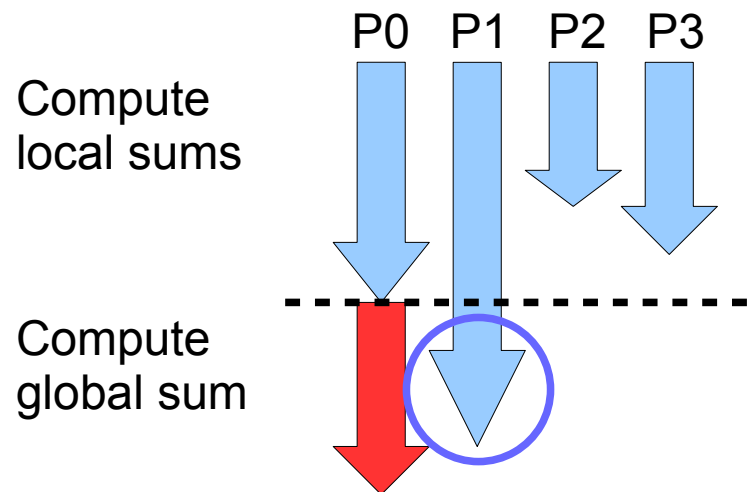
(wrong, in a subtle way)

- We use a shared array `psum[]` where each processor can store its local sum
- At the end, one processor computes the global sum

```
psum[0..P-1] = 0.0; /* all elements set to 0.0 */ WRONG
sum = 0.0;
do in parallel {
    my_start = (n * my_id) / P;
    my_end = (n * (my_id + 1)) / P;
    for (my_i=my_start; my_i<my_end; my_i++) {
        psum[my_id] += v[my_i];
    }
    if ( 0 == my_id ) { /* only the master executes this */
        for (my_i=0; my_i<P; my_i++)
            sum += psum[my_i];
    }
}
```

The problem with version 5

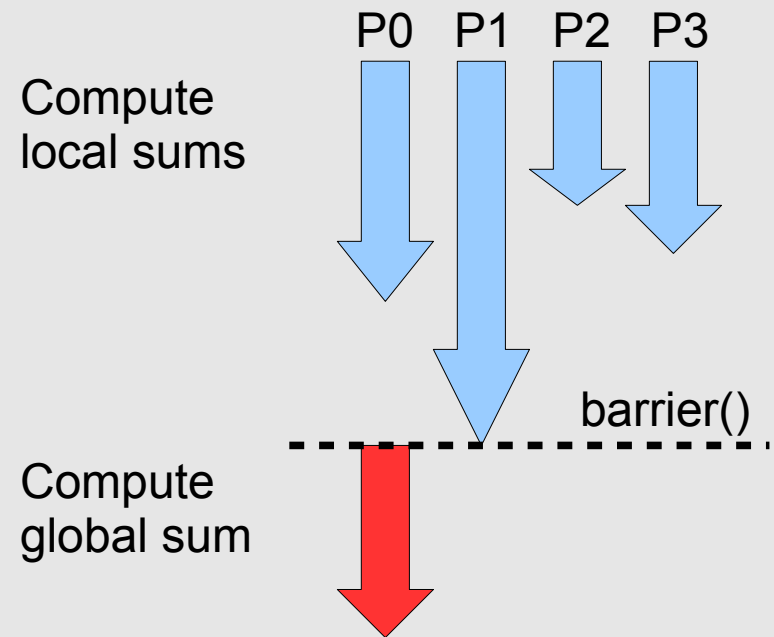
- Processor 0 could start the computation of the global sum before all other processors have computed the local sums!



Version 6 (correct)

- Use a **barrier synchronization**

```
psum[0..P-1] = 0.0;
sum = 0.0;
do in parallel {
    my_start = (n * my_id) / P;
    my_end = (n * (my_id + 1)) / P;
    psum[0..P-1] = 0.0;
    for ( my_i=my_start;
          my_i<my_end; my_i++ ) {
        psum[my_id] += v[my_i];
    }
    barrier();
    if (my_id == 0) {
        for (my_i=0; my_i<P; my_i++)
            sum += psum[my_i];
    }
}
```



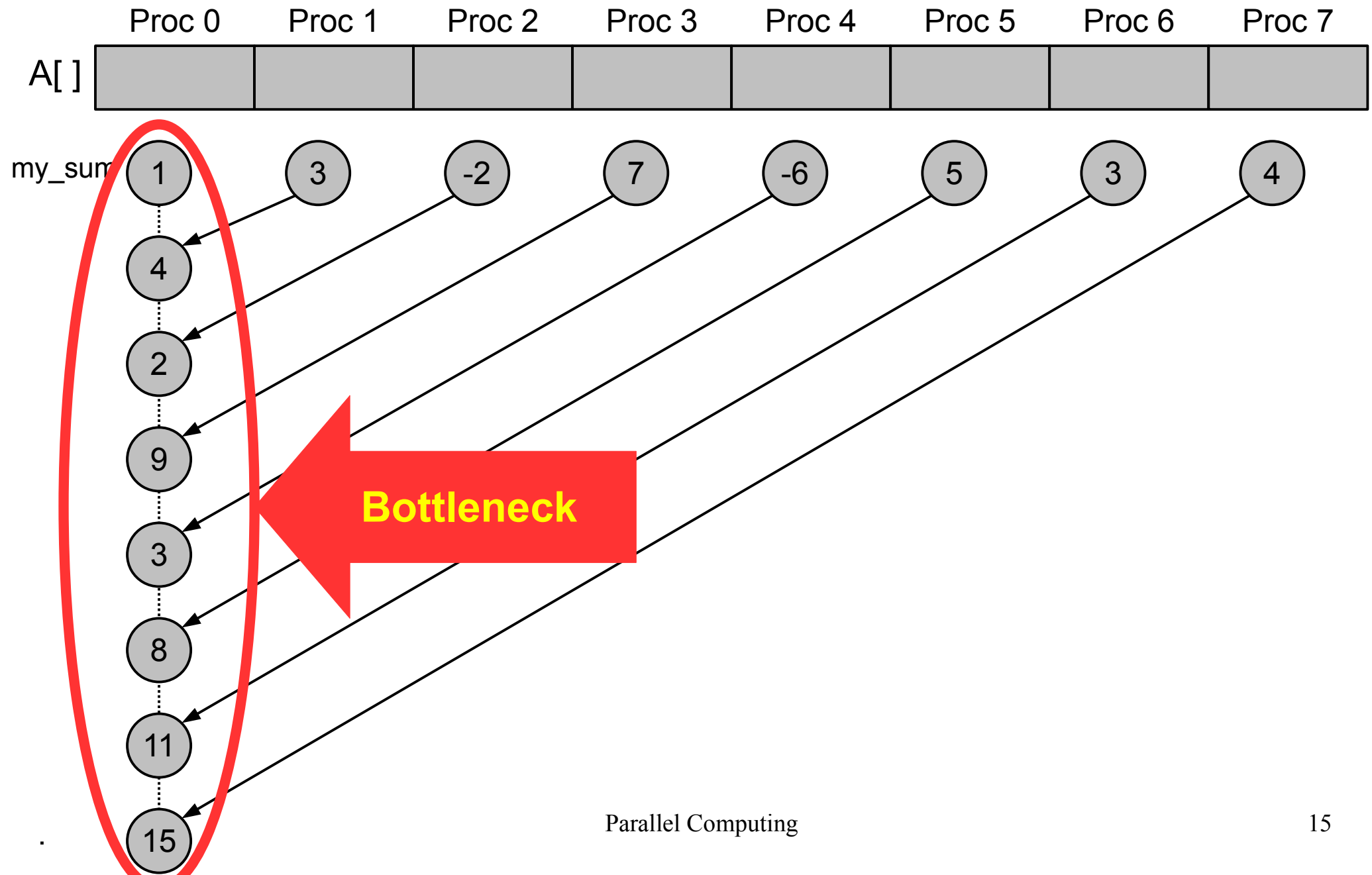
Distributed-memory version

- $P \ll n$ processors
- All variables are private
- Each processor computes a local sum
- Each processor sends the local sum to processor 0 (the master)

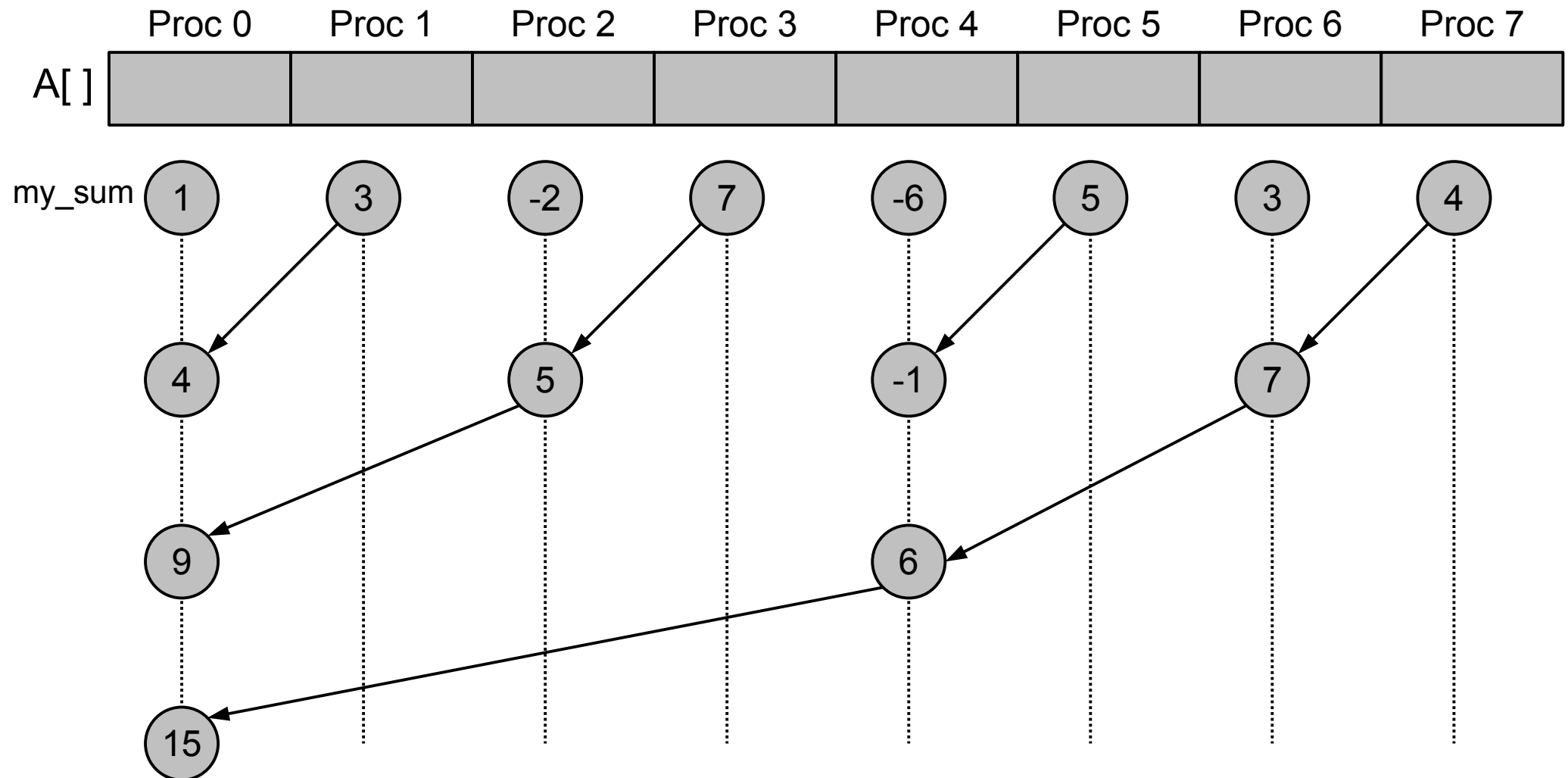
Executed by process my_id:

```
...
my_sum = 0.0;
my_start = ..., my_end = ...;
my_v[] = receive v[my_start..my_end-1]
from proc 0;
for ( i=0; i < my_end-my_start; i++ ) {
    my_sum += my_v[i];
}
if ( 0 == my_id ) {
    for ( i=1; i<P; i++ ) {
        tmp = receive from proc i;
        my_sum += tmp;
    }
    printf("The sum is %f\n", my_sum);
} else {
    send my_sum to proc 0;
}
```

Distributed-memory version



Parallel reduction



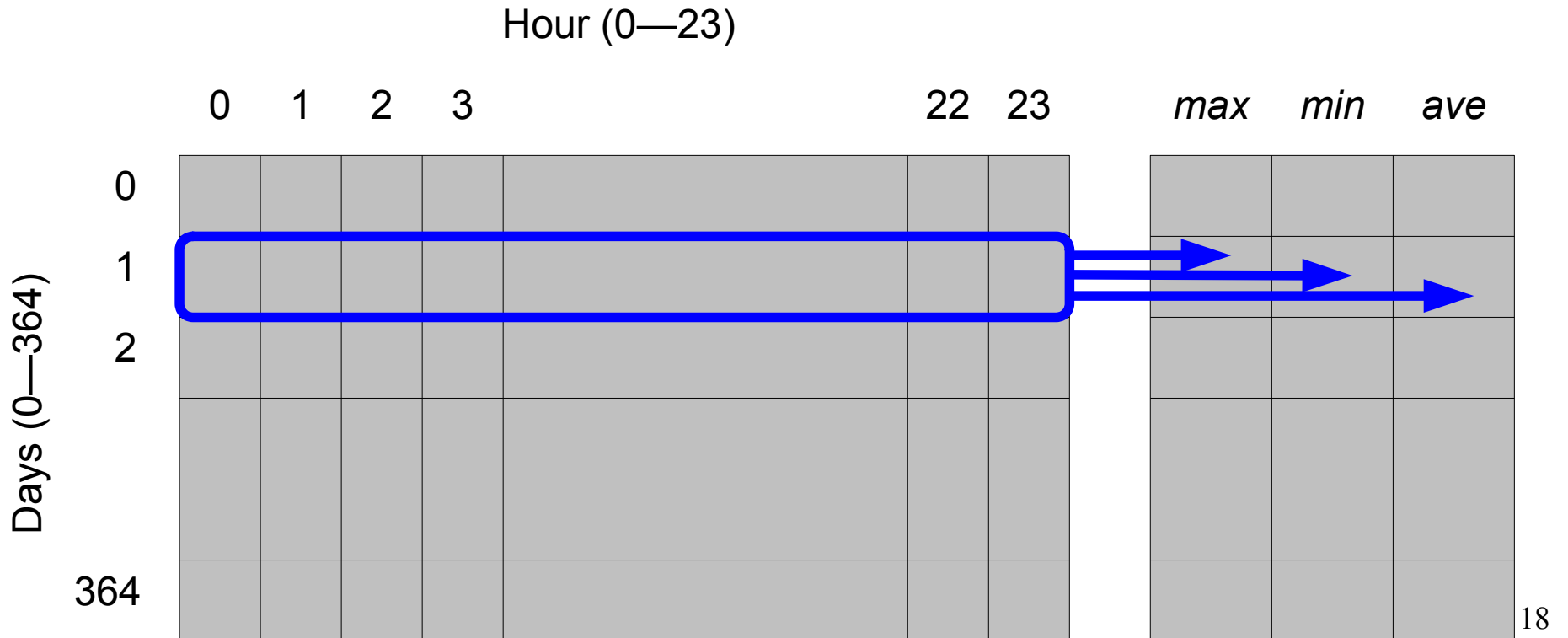
- $(P - 1)$ sums are still performed; however, processor 0 receives $\sim \log_2 P$ messages and performs $\sim \log_2 P$ sums

Task parallelism vs Data parallelism

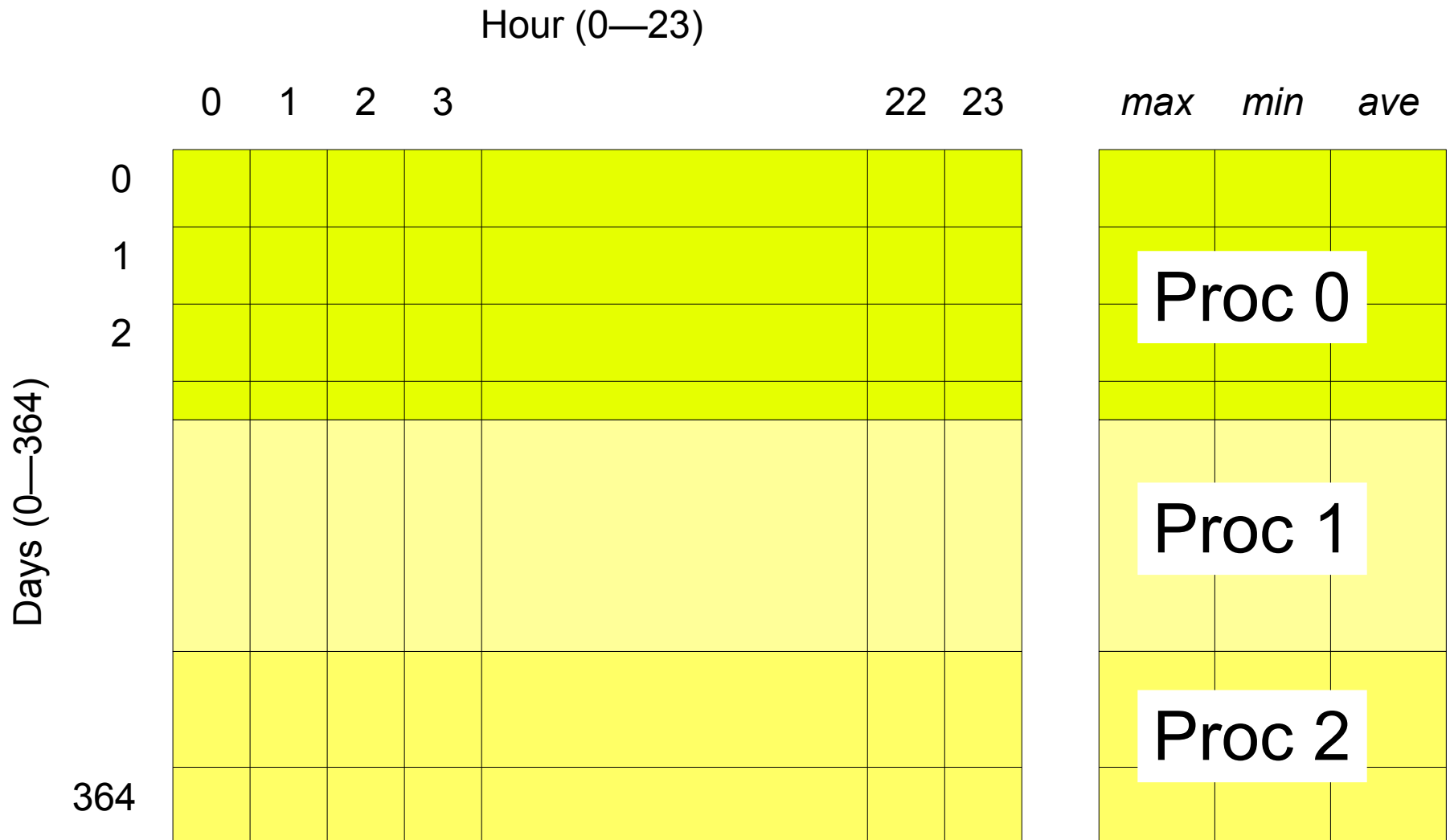
- Task Parallelism
 - Distribute (possibly different) tasks to processors
- Data Parallelism
 - Distribute data to processors
 - Each processor executes the same task on different data

Example

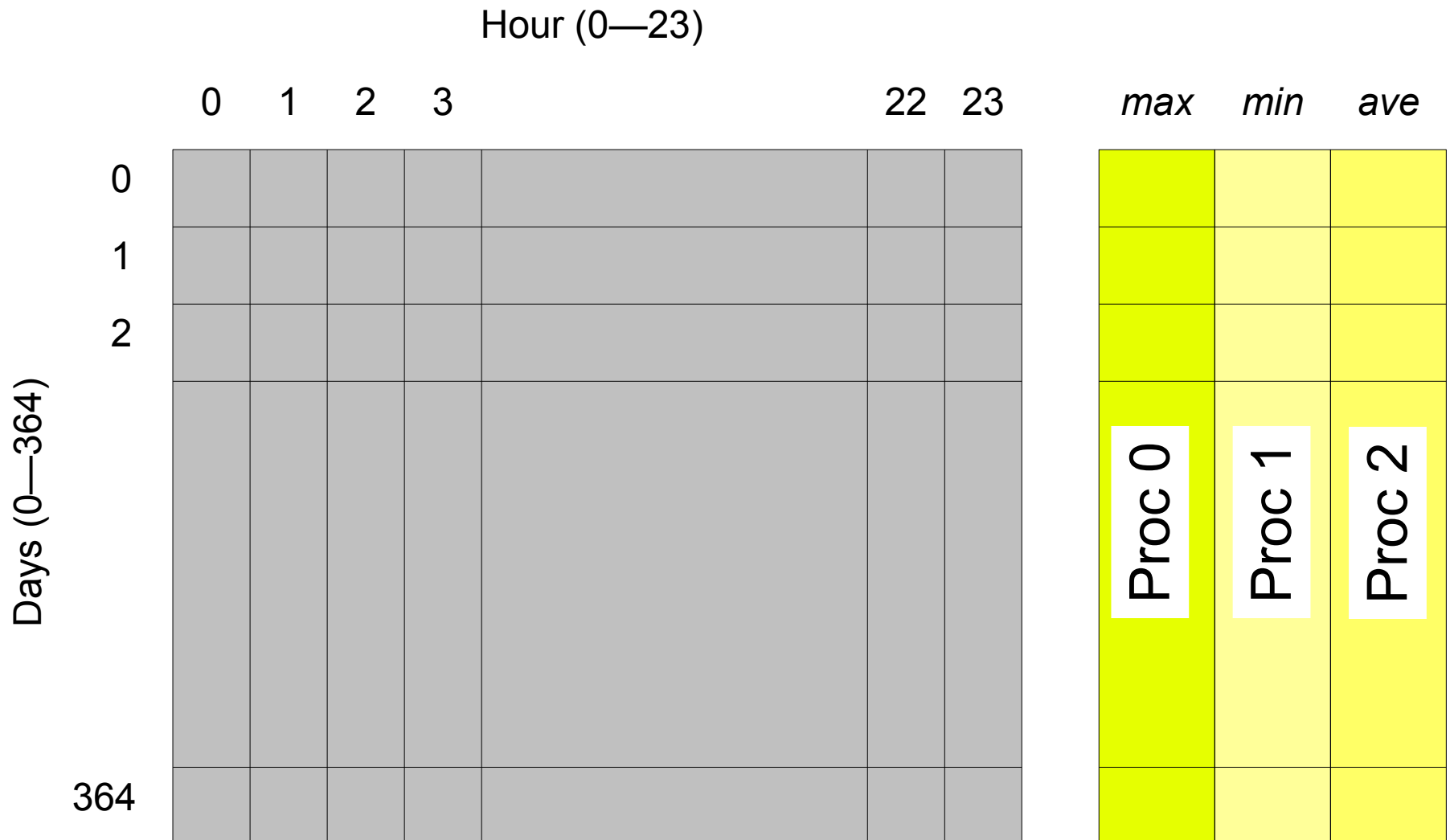
- Table containing hourly temperatures on some location
 - 24 columns, 365 rows
- Compute daily minimum, maximum and average
- Assume 3 execution units



Data parallel approach



Task parallel approach



Introduction to Data Parallel Supercomputing

Thinking Machines Corporation (1989) , Introduction to Data Parallel Supercomputing, https://www.youtube.com/watch?v=f8NTHfMw_WA

Key concepts

- Parallel architectures require parallel programming paradigms
- Writing parallel programs is harder than writing sequential programs